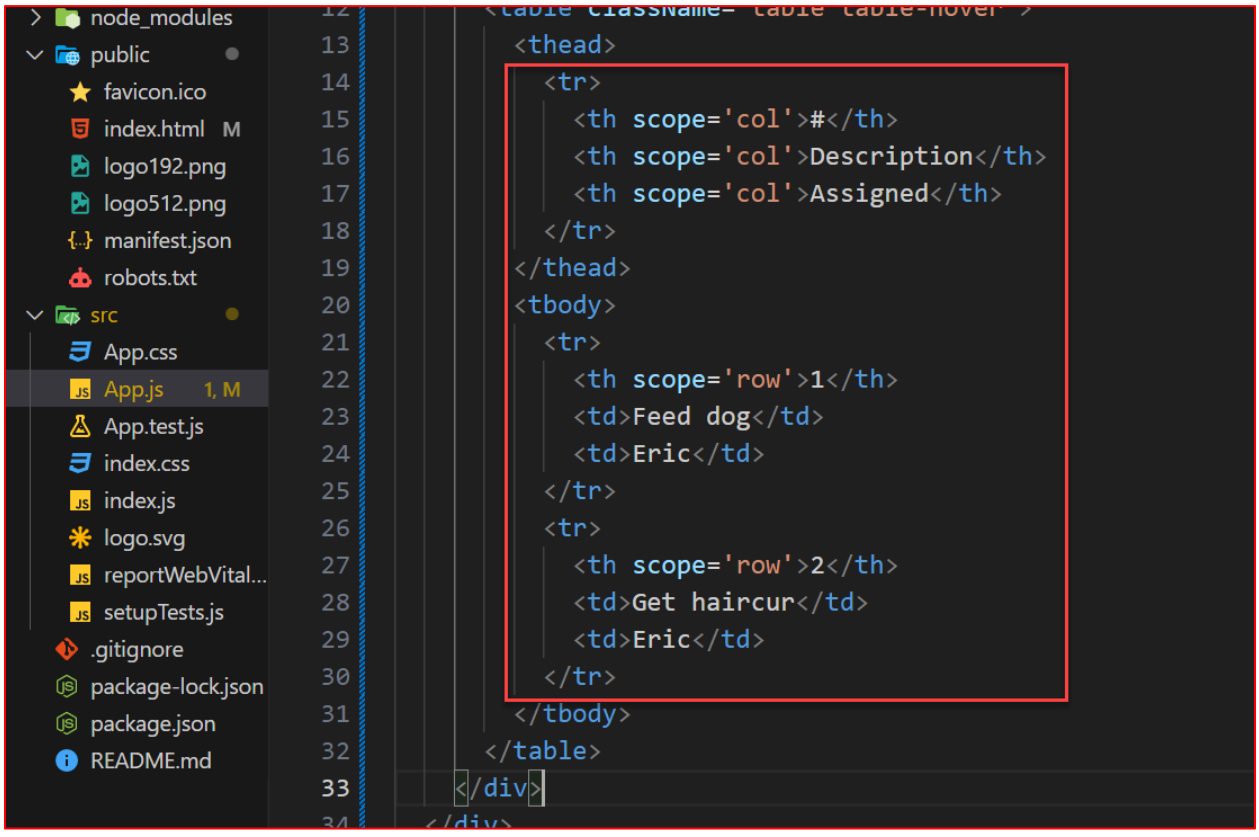
# Creating a New React Component

As we continue our journey with our React application for managing to-dos, we are now entering an exciting phase: the creation of our first child component within our `App.js` file.

## Understanding the Structure

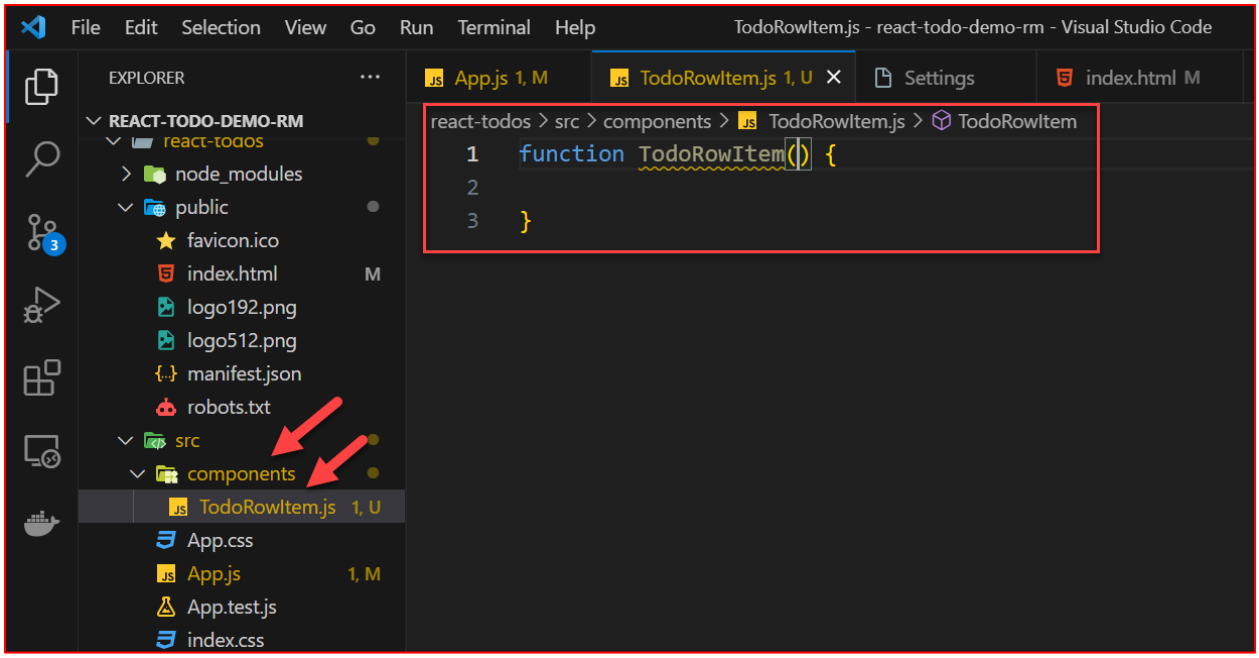
When we examine our `App.js` file, we encounter a lengthy DOM structure. From the initialization of the entire page with our container to defining cards with headers and bodies, we've constructed the entire functionality of our to-do application.

Specifically, we have a table within the card body that holds our entire to-do list. This is where we will focus our attention by creating our first component to represent individual table rows. These rows are crucial because they contain the precise details of our to-dos, including their descriptions and assignments.



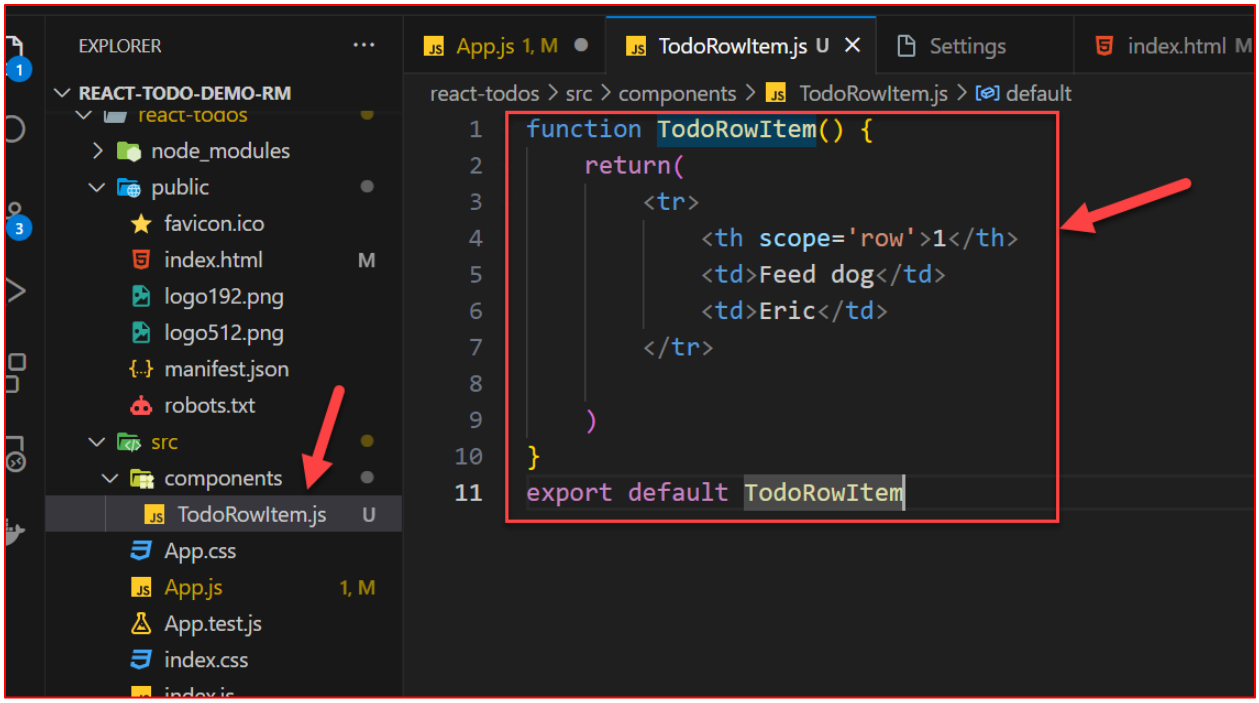

## Creating a New Component

Let's start by creating a new folder in our source directory. Right-click and select "New Folder," naming it "components." Inside this folder, we'll create a new JavaScript file named `TodoRowItem.js`.




Here's a step-by-step breakdown of what we'll do:

1. **Initialize the File:** You'll begin with an empty JavaScript file. Start by typing a function named `TodoRowItem`, followed by parentheses, like any other function, enclosed in brackets.
2. **Exporting the Component:** Outside the brackets, add `export default TodoRowItem`. This command prepares our component for use within our application, though our function isn't doing anything just yet.
3. **Returning HTML:** Since we need this function to return the HTML DOM that should be rendered, let's say `return`, followed by parentheses.
4. **Defining the Table Row:** We must return a single element. If we wanted to create multiple table rows, we could enclose them in a `div`, satisfying the requirement for a single parent. However, in this case, we'll return a single table row with specific data. For example, a scope row of one, table data of "feed dog," and another table data that says "Eric."

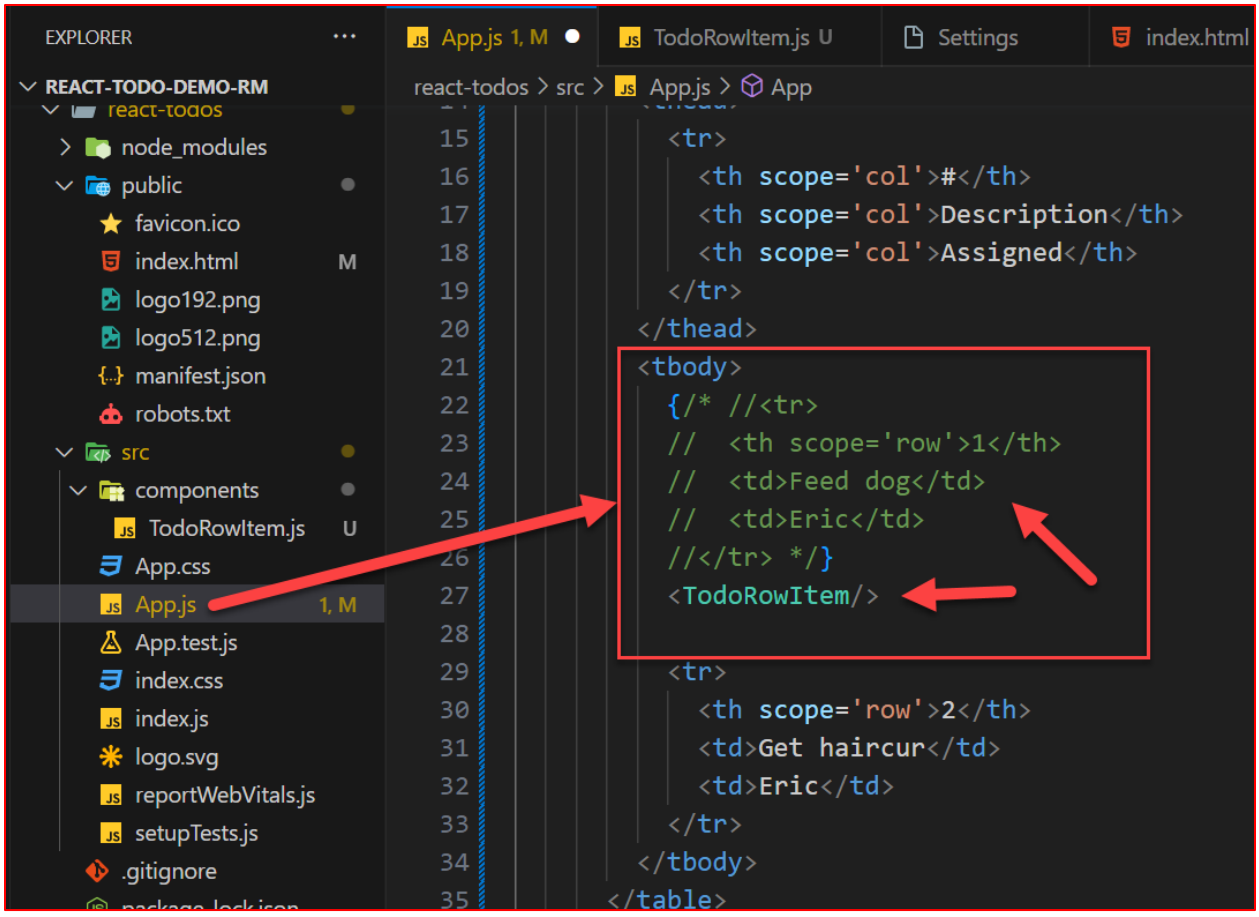



## Implementing the Component

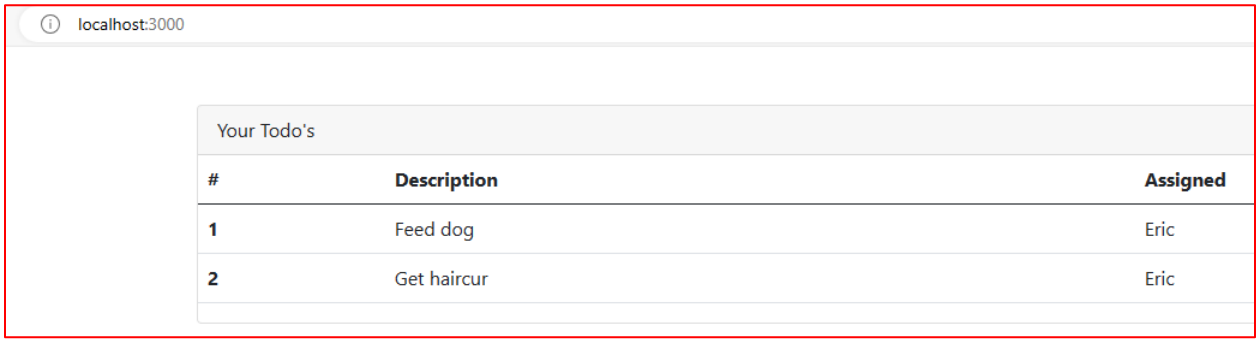
Now we're returning the `TodoRowItem`, which holds this data. Back in our `App.js` file, we need to import our new component by typing `import TodoRowItem from './components/TodoRowItem'`.




Next, locate the existing table row that includes "Feed Dog" and "Eric," delete it, and replace it with our new component, `TodoRowItem`. You'll notice that we use normal HTML tags as we would with any other HTML element.




After saving and refreshing the browser, it might seem that nothing has changed, but that's because we've passed the exact same data through the component.




Try altering the text slightly, such as spelling "dog" with two G's, and you'll see the change reflected in the browser.

## Summary

You've just created and rendered your first component within `App.js`, dynamically updating the DOM. This process of component creation and integration is fundamental to React development, allowing for modular, reusable code that enhances maintainability and readability.